

Secure Notes Web Application

EthicalHackingand SecurityEvaluationReport

AhmedAtaf 22102274

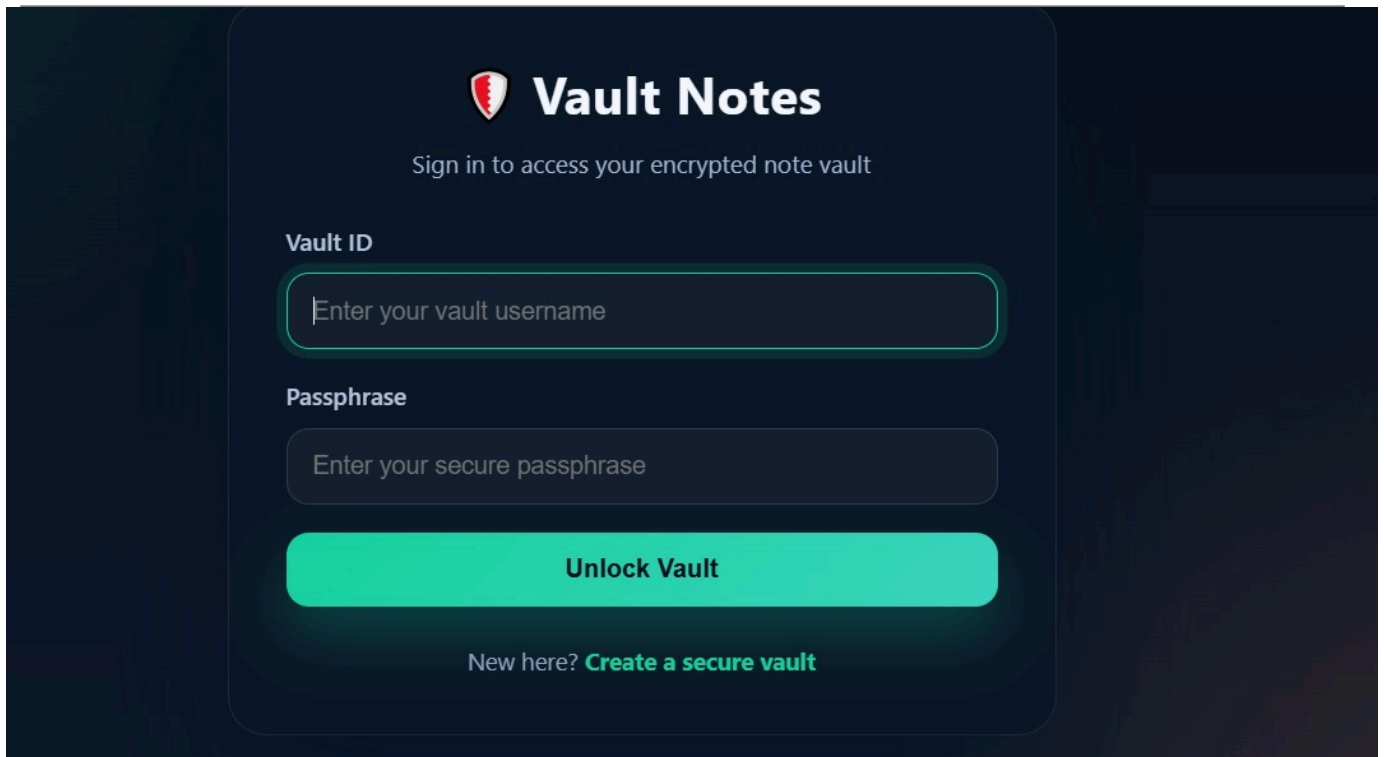
Abdelrahman Ahmed 23101413

Ahmed Fekry 23101358

1. System Description

The developed system is a simple secure web application called Secure Notes. The application allows users to create an account, log in, and store private notes. The project was intentionally kept simple so that the focus is on secure coding, vulnerability testing, and understanding how common web attacks work against a controlled application.

The application was built using Python Flask for the backend, SQLite for the database, HTML templates for the interface, and CSS for basic styling. Users must register with a valid username and a strong password. After logging in, each user can add notes that are saved in the database and displayed only to that user.



The screenshot displays the login page for 'Vault Notes'. At the top, there is a shield icon with a red and white design, followed by the text 'Vault Notes' in a large, bold, white font. Below this, a subtitle reads 'Sign in to access your encrypted note vault'. The form consists of two input fields: 'Vault ID' with a placeholder 'Enter your vault username' and 'Passphrase' with a placeholder 'Enter your secure passphrase'. Both fields have a green border. Below the input fields is a large, rounded rectangular button with a green gradient and the text 'Unlock Vault'. At the bottom, there is a link that says 'New here? Create a secure vault'.

Add a new vault entry

Enter your note for safekeeping...

Secure Note

Your vault is empty. Add a secure note above to get started.

2. Security Implementation

Several security mechanisms were implemented to protect the application from common web vulnerabilities.

Password hashing: User passwords are not stored in plain text. The application uses bcrypt to hash passwords before saving them in the database.

Server-side validation: Usernames and passwords are validated on the server, not only in the browser. Notes are also limited to valid length.

SQL Injection protection: Database operations use parameterized queries with placeholders instead of directly inserting user input into SQL statements.

XSS protection: The application uses Flask/Jinja autoescaping so that user-submitted note content is displayed as text instead of executable HTML or JavaScript.

Login attempt limitation: The system locks an account temporarily after repeated failed login attempts to reduce brute-force and dictionary attack risk.

3. Attacks Performed

3.1 SQL Injection Attack

A SQL Injection attack was tested by entering a malicious username such as ' OR '1'='1 in the login form. The purpose of this attack was to trick the database into bypassing authentication by changing the meaning of the SQL query.

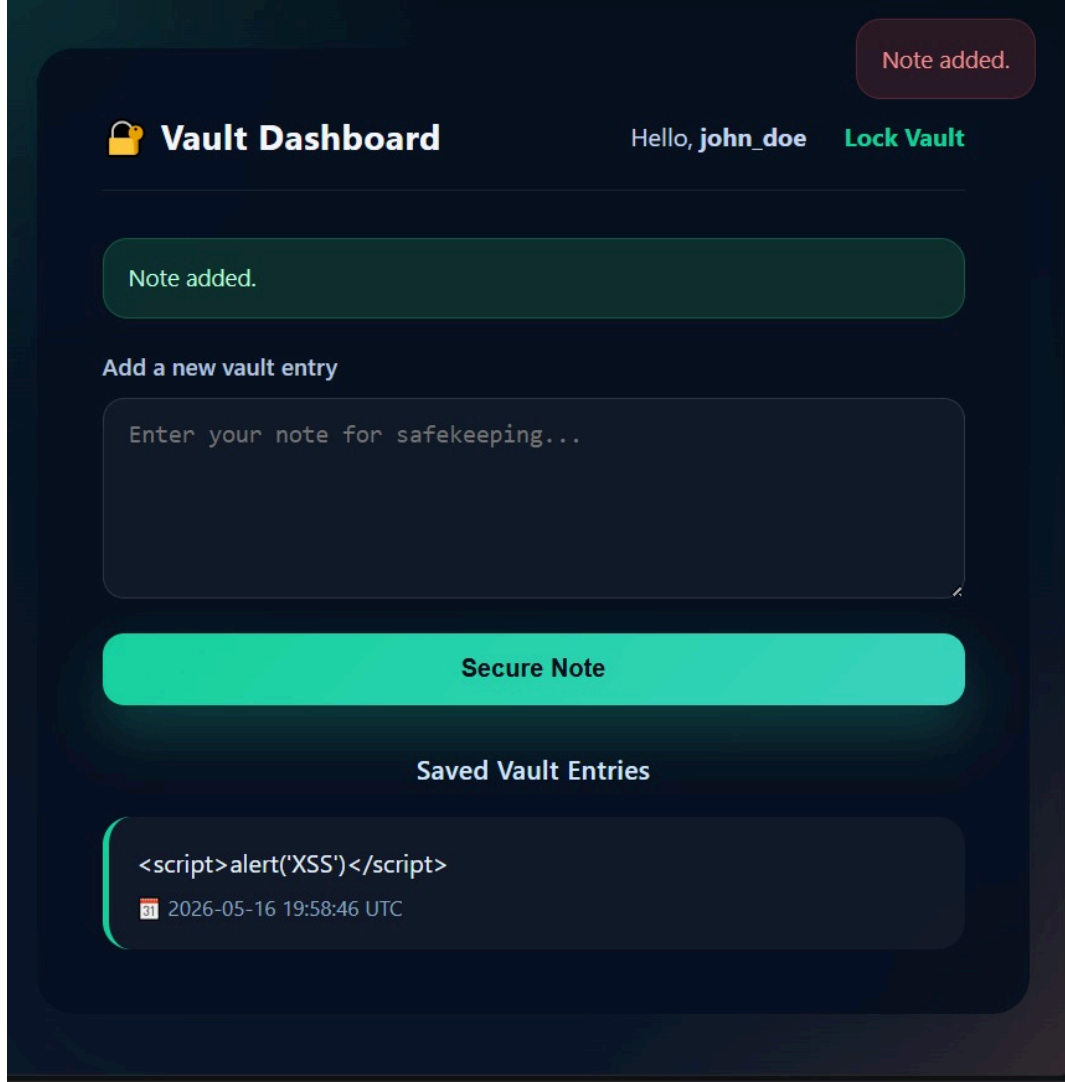
Expected result: The attack should fail because the application uses parameterized queries. The malicious input is treated as normal text rather than executable SQL code.

The screenshot shows the 'Vault Notes' login page. At the top right, a red error message reads 'Invalid username or password.' Below the 'Vault Notes' logo and sign-in prompt, another red error message 'Invalid username or password.' is displayed above the 'Vault ID' input field. The 'Vault ID' field contains the payload `' OR '1'='1`. The 'Passphrase' field is filled with dots. A large green 'Unlock Vault' button is at the bottom, with a link 'New here? Create a secure vault' below it.

3.2 Cross-Site Scripting (XSS) Attack

A stored XSS attack was tested by adding a note containing a script payload such as `<script>alert('XSS')</script>`. The goal of this attack was to make the browser execute JavaScript when the note is displayed.

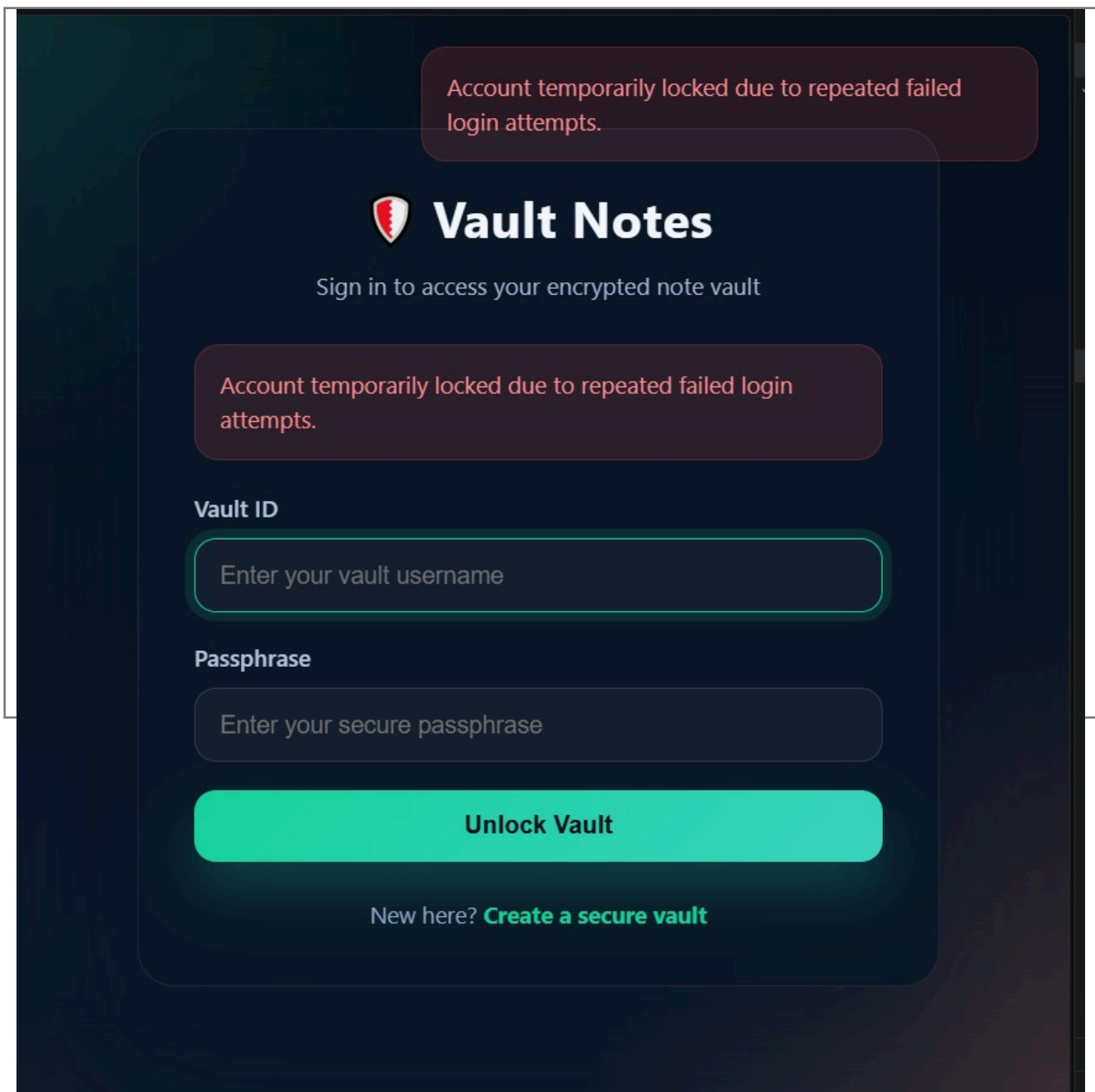
Expected result: The attack should fail because the template engine escapes user input. The script should appear as plain text and should not run in the browser.



3.3 Brute Force Attack

A brute-force attack was tested by repeatedly trying different incorrect passwords for the same account. The purpose of this attack was to see whether unlimited login attempts could eventually allow an attacker to guess the password.

Expected result: The attack should fail or be slowed down because the system tracks failed login attempts and temporarily locks the account after repeated failures.



3.4 Dictionary Attack

A dictionary attack was tested by trying common password guesses such as password, 12345678, qwerty, admin123, and other weak passwords. This attack checks whether weak or predictable passwords can be guessed easily.

Expected result: The attack should fail because the registered password follows a stronger password policy and because repeated failed attempts can trigger account lockout.

Password must be at least 8 characters and include uppercase, lowercase, number, and symbol.



Create Vault

Build a new secure note vault for your private data

Password must be at least 8 characters and include uppercase, lowercase, number, and symbol.

Vault Username

Secure Passphrase

Create Vault

Already have a vault? [Sign in](#)

4. Results and Analysis

Attack	Result	Reason	Security Control
SQL Injection	Failed	The malicious SQL payload was treated as text and did not change the database query.	Parameterized queries
XSS	Failed	The script payload was escaped and displayed as text instead of running in the browser.	Output escaping / Jinja autoescaping
Brute Force	Failed / Limited	Repeated wrong passwords triggered temporary account lockout.	Login attempt limitation
Dictionary Attack	Failed / Limited	Common weak guesses did not match the strong password and repeated failures were limited.	Strong password policy and lockout

5. Mitigation Techniques

The following mitigation techniques were used to prevent or reduce the tested vulnerabilities:

Password hashing with bcrypt prevents attackers from seeing real passwords even if the database is exposed.

Parameterized queries prevent SQL Injection by separating SQL code from user input.

Server-side validation rejects invalid usernames, weak passwords, and overly long note content.

Output escaping prevents stored XSS by making HTML and JavaScript payloads display as text.

Login attempt limitations reduce the effectiveness of brute-force and dictionary attacks by temporarily locking accounts after repeated failed attempts.

Strong password requirements reduce the chance of users choosing common or easily guessed passwords.

6. Ethical Considerations

All testing activities were conducted solely on our own application in accordance with ethical and legal guidelines. No third-party systems, websites, accounts, or networks were targeted. The purpose of the testing was educational: to understand vulnerabilities, evaluate defenses, and improve the security of the developed application.

7. Conclusion

This project demonstrated how a basic web application can be built and secured against common vulnerabilities. The testing showed that SQL Injection and XSS attacks can be prevented through parameterized queries and output escaping. The brute-force and dictionary attack tests showed the importance of login attempt limits and strong password policies. Overall, the project improved understanding of secure coding, ethical hacking, vulnerability testing, and professional security documentation.

8. Appendix: How to Run and Test the Project

To run the application, install the required packages using `pip install -r requirements.txt`, then run `python app.py`. Open `http://127.0.0.1:5000` in a browser. After creating an account, perform the required tests manually or run `python attack_tests.py` while the Flask server is running.

Recommended screenshots to add before final submission: registration page, successful login dashboard, SQL Injection payload and failed result, XSS payload displayed safely, repeated failed login attempts, lockout message, and dictionary attack results.